Software Requirements Specification

BeerFinder API
Version 1.0.0

BeerFinder API Team

Revisions

Version	Primary Authors	Description of Version	Date completed
1.0.0	Claire Kayton, Sarah Hathaway, Liam Linenberger, Roger Chang, and Matt Thornton	Creation of the primary version of this design document.	10/15/2025
1.0.1	Claire Kayton, Sarah Hathaway, Liam Linenberger, Roger Chang, and Matt Thornton	Updated SRS and data flow diagram to better match the demo we are delivering.	12/8/2025

Table of Contents

1. Introduction

1.1 Purpose

This document outlines the assumptions, constraints, and requirements of a Beer Finding API. This document is intended to serve as a reference for Claire Kayton, Sarah Hathaway, Liam Linenberger, Roger Chang, Matt Thornton, and the project developers, as well as for potential future developers.

1.2 Scope

The scope of this system is to build an API that could communicate between a website and a datastore, allowing users to search for local Colorado beers, and also allowing breweries to add, update, and delete their own beers in the datastore.

1.3 Definition, Acronyms, and Abbreviations

- API - Application Programming Interface
- User - website user
- Brewery - A 3rd party company that passes registration and can input their products in our datastore.
- JDBC - Java Database Connectivity
- RESTful - (Representable State Transfer) A set of rules and conventions about how web systems should interact.

2. Overall Description

2.1 Product Perspective

The **BeerFinder API** is a RESTful API designed to act as the central data and logic layer within a larger ecosystem of beer-related applications. It provides a unified interface for querying, filtering, and managing information about beers, breweries, styles, and availability across multiple locations within Colorado.

The API can handle HTTP requests and access a datastore via JDBC.

2.1.1 Software Interfaces

- Datastore through JDBC

- HTTP Requests

2.1.2 Communication Interfaces

- TCP-IP
- SSL (Optional)

2.2 Product Functions

2.2.1 API Function

- API connects to a BeerFinder Website
- API connects to a datastore
- API receives HTTP requests and returns data to a BeerFinder Website
- API receives data from Breweries and stores that data in a Datastore

2.3 Constraints

- Access via JDBC.

2.4 Assumptions and Dependencies

- Any User will go through the API.
- There must be a Datastore to request and store data.

3. Specific Requirements

3.1 External Interfaces

3.1.1 Server Interfaces

- Connection to Datastore through JDBC.

3.1.2 HTTP Requests

GET	
/api/beers	Returns all beers.
/api/beers/{id}	Returns beer with a specific id.
/api/breweryPage/{breweryId}	Returns all beers associated with breweryId.

POST	
/api/brewerySignIn	Send a package with user + password to sign in.
/api/breweryPage/{breweryId}	Send a package with beerId, name, style, and abv to make new beers.

PUT	
/api/breweryPage/{breweryId}	Send a package with beerId, name, style, and abv to update existing beers.

DELETE	
/api/breweryPage/{breweryId)	Send a package with beerId to delete that beer.

3.2 Functional Requirements

- All user input shall be validated via an API.
- API will use a RESTful design.

3.3 Performance Requirements

- All searches shall return some form of information, whether it be an error or a successful search.

3.4 Security Requirements

- Breweries shall provide a username and a password for authentication.
- Brewery information, such as username and password, is redacted when regular users search for beers.

3.5 Data Requirements

- Shall store up to 100,000 users.
- Datastore shall store Brewery information.
- Datastore shall store Beer information.
- Every beer shall have a corresponding Brewery

4. Appendix

4.1 GitHub

IntelliJ Code
https://github.com/clairek1786/BeerFinderAPI

4.2 “Parking Lot”

This section is for known issues and features that need to be added.

- Requests can be sent to a brewery as long as the brewery ID is known and manually entered into the URL under any field requiring it.